```
Name inside method1
Anand
Name inside method2:
Ram
Name inside method1:
Anand
Name outside methods: Anand
```

Looping in Ruby

In Ruby, loops help in the execution of the same code block, a number of times. The *while* loop, one of the most used in programming languages, helps in executing a code block when a given condition is true.

The syntax is:

```
while condition [do]
   code
end
```

…or:

```
code while condition
```

…or:

```
begin
  code
end while condition
```

An example is:

```
$j = 0
$num = 10
while $j < $num  do
   puts "Inside loop j = #$j"
   $j +=1
end
```

The *until* statement: Unlike the *while* statement, the *until* statement helps in the execution of a block of code when the condition is false.

The syntax is:

```
until conditional [do]
   code
end
```

…or:

```
code until conditional
```

…or:

```
begin
   code
end until conditional
```

An example is:

```
$j = 0
$num = 5
until $j > $num  do
   puts "Inside the loop j = #$j"
   $j +=1;
end
```

The *for* loop: A *for* statement in Ruby is used for the execution code, once for each of the elements in an expression.

The syntax is:

```
for variable [, variable ...] in expression [do]
   code
end
```

For example:

```
for a in 0..5
   puts "Value of variable is #{a}"
end
```

The *break* statement: A *break* statement helps in the termination of an internal loop. Here is an example:

```
for a in 0..5
   if a > 2 then
      break
   end
   puts "Value of the variable is #{a}"
end
Output
Value of the variable is 0
Value of the variable is 1
Value of the variable is 2
```

The *next* statement: A *next* statement helps in jumping to the next iteration of the most internal loop. For example:

```
for a in 0..5
   if a < 2 then
      next
   end
   puts "Value is #{a}"
end
Output
Value is 2
Value is 3
Value is 4
Value is 5
```